

EDINET APIとXBRLデータ解析に関する包括的技術リサーチ報告書

EDINET API v2 の仕様詳細と実務運用

金融庁が提供する有価証券報告書等の開示書類を電子的に取得するためのRESTfulなAPIであるEDINET APIは、2024年4月にVersion 2へと刷新された¹。APIを利用するためには事前に登録を行い、APIキーを取得する必要があるが、このキーの発行および利用自体は完全な無料枠で提供されており、個人開発者であっても障壁なく利用を開始できる²。

書類一覧を取得するためのエンドポイントである /api/v2/documents.json では、指定した日付に提出された書類のメタデータ一覧をJSON形式で一括取得することが可能である¹。パラメータとして、当日から5年前までの日付を YYYY-MM-DD 形式で指定することができ、企業の属性情報や書類の固有IDである docID を特定するために最初に叩くべきリクエストとなる²。レスポンスには、提出者のEDINETコード、証券コード、提出日時、書類種別コード(docTypeCode)、府令コード(ordinanceCode)、様式コード(formCode)などのメタデータが配列として格納されており、この情報をもとに必要な書類を絞り込む⁴。

書類データを取得するためのエンドポイントは /api/v2/documents/{docID} であり、一覧APIから取得した docID をパスパラメータに指定して実データを取得する³。Version 2における書類取得APIでは type パラメータが重要であり、取得するファイル形式を以下のように制御する³。

type パラメータ	取得可能なデータ形式	特徴および用途
type=1	XBRLファイル (ZIP形式)	財務諸表の本表や注記を含む完全な構造化データ ⁶
type=2	PDFファイル	閲覧および印刷用のレイアウトが保持されたドキュメント ⁶
type=5	CSVファイル (ZIP形式)	勘定科目のサマリーや主要財務数値が抽出された軽量データ ³

株式投資分析において極めて重要となる有価証券報告書のみを適切にフィルタリングするためには、レスポンスに含まれる書類種別コード等の3つを論理積で評価する必要がある⁴。具体的には、docTypeCode が "120" であり、かつ ordinanceCode が "010"、formCode が "030000" であるものが通常の有価証券報告書に該当する⁴。

APIのレート制限については、公式ドキュメント上で厳格なリクエスト秒数や回数上限が明文化されて

いないものの、一般的なREST APIの運用ベストプラクティスや先行する開発コミュニティの事例に基づき、1秒あたり1から2リクエスト程度に抑えてアクセスを行うことが推奨される(信頼度レベル:高)。過度なバーストアクセスはサーバーからの拒絶やIP制限を招く要因となり、特に無料のサーバーレス環境から大量のリクエストを並列で発行する設計は避けるべきである。Version 2より本格的に実用化された type=5 (CSV形式) のデータは、従来の巨大なXBRLを直接解析する手間を劇的に削減できるため、特定の主要財務項目のみを即座に抽出してデータベースに格納する目的においては非常に高い実用性を持つ²。

XBRL データの構造と解析における落とし穴

XBRLデータの内部構造を理解することは、解析処理を正しく設計する上で極めて重要である。EDINETから type=1 でダウンロードされるZIPファイルを展開すると、通常は PublicDoc もしくは XBRL といった規定のディレクトリ構造が出現する⁷。主要な財務報告データや提出書類の本体、インラインXBRLファイルおよびタクソノミ定義ファイルは PublicDoc フォルダ内に格納されており、この配下にあるファイルを解析対象とすることになる⁷。

データの記述方式には、従来のピュアなXMLベースのXBRL形式(.xbrl)と、人間がブラウザ等で閲覧する際の視覚的な整合性を維持するためにHTMLの中にXBRLのタグを埋め込んだインラインXBRL(iXBRL、.htm)の2種類が存在する⁸。現在のEDINETにおける提出書類では、有価証券報告書の主要な部分において、このiXBRL形式が標準的に採用されている(信頼度レベル:高)。

解析を行う際には、XMLの属性として定義されている名前空間プレフィックスの役割を正確に把握する必要がある¹¹。これらはデータの分類や意味を定義するタクソノミとして機能しており、多岐にわたる報告要件に対応している¹¹。

プレフィックス	役割・定義	該当する主なデータ項目
jpcrp_cor	企業内容等の開示に関する府令に基づくタクソノミ ¹¹	事業の内容、従業員の状況、セグメント情報など
jppfs_cor	財務諸表の表示に関するタクソノミ(日本基準) ⁵	貸借対照表、損益計算書、キャッシュ・フロー計算書
jpdei_cor	提出書類の基本情報(DEI)を定義するタクソノミ ¹¹	会社名、会計基準、決算日、発行済株式数 ¹⁵
jpigp_cor	国際会計基準(IFRS)に基づくタクソノミ ¹¹	IFRS採用企業における財政状態計算書、損益計算書 ¹³

XBRL解析における最大の難所の1つは、コンテキスト(contextRef)の解釈である¹⁷。1つのXBRLファイルの中には、連結決算の数値、単体決算の数値、当期の数値、前期の数値、あるいは四半期累計の数値などが混在して格納されている¹²。これらを正しく識別するためには、contextRef 属性の文

字列に含まれる識別子を評価する必要がある¹⁷。例えば、単体決算の数値を特定したい場合には、文字列中に NonConsolidatedMember などの文言が含まれているかどうかを判定条件として組み込むことが不可欠となる¹²。

さらに、iXBRL形式のデータにおいて極めて重要な属性が scale である¹⁸。この属性は、数値データの桁数を指定するものであり、例えば scale="6" と指定されている場合は、記載されている数値が「100万」単位であることを示している¹⁷。したがって、パースした生データに対して 10^{scale} を乗算する処理を行わなければ、実際の金額と100万倍の乖離が生じることになり、財務分析アプリとしては致命的な誤りとなる¹⁸。scale の処理を忘れること、あるいは scale が設定されていない項目に対しても一律で処理を適用してしまうことは、XBRL解析における典型的な陥りやすい罠である。

主要財務項目のXBRLタグ名(3基準対照)

日本の株式市場に上場する企業は、自社が採用している会計基準(日本基準(J-GAAP)、国際会計基準(IFRS)、米国基準(US-GAAP))に従って有価証券報告書を提出する。したがって、同一の財務項目であっても、基準ごとに参照すべきXBRLのタグ名(要素名)が異なる。以下に、主要な財務データについて、各会計基準における代表的なタグ名を整理する。なお、実務においては各企業のタクソノミ拡張等によって多少の表記揺れが生じる場合がある(信頼度レベル:中)。

財務項目	日本基準(J-GAAP)	国際会計基準(IFRS)	米国基準(US-GAAP)
売上高	jppfs_cor:NetSales または jppfs_cor:Revenues	jpigp_cor:NetSalesIFRS または jpigp_cor:RevenueIFRS ¹⁰	us-gaap:Revenues または us-gaap:SalesRevenueNet
営業利益	jppfs_cor:OperatingIncome	jpigp_cor:OperatingProfitLossIFRS ¹⁶	us-gaap:OperatingIncomeLoss
当期純利益(親会社帰属)	jppfs_cor:ProfitLossAttributableToOwnersOfParent	jpigp_cor:ProfitLossAttributableToOwnersOfParentIFRS ¹⁶	us-gaap:NetIncomeLossAvailableToCommonStockholders
総資産	jppfs_cor:TotalAssets	jpigp_cor:TotalAssetsIFRS	us-gaap:AssetsTotal
自己資本/純資産	jppfs_cor:TotalNetAssets	jpigp_cor:TotalEquityIFRS	us-gaap:StockholdersEquity

営業CF	jppfs_cor:NetCashProvidedByUsedInOperatingActivities ⁵	jpigp_cor:NetCashProvidedByUsedInOperatingActivitiesIFRS	us-gaap:NetCashProvidedByUsedInOperatingActivities
投資CF	jppfs_cor:NetCashProvidedByUsedInInvestmentActivities ⁵	jpigp_cor:NetCashProvidedByUsedInInvestmentActivitiesIFRS	us-gaap:NetCashProvidedByUsedInInvestingActivities
発行済株式数	jpdei_cor:TotalNumberOfIssuedSharesDEI	jpdei_cor:TotalNumberOfIssuedSharesDEI	us-gaap:CommonStockSharesIssued
EPS	jppfs_cor:BasicEarningsPerShare	jpigp_cor:BasicEarningsPerShareIFRS	us-gaap:EarningsPerShareBasic
支払利息	jppfs_cor:InterestExpenses	jpigp_cor:InterestExpenseIFRS	us-gaap:InterestExpense

有利子負債については、J-GAAP等において単一の決定的なタグが定義されていないことが多く、短期借入金、1年以内返済の長期借入金、長期借入金、社債などの各タグを個別に抽出し、プログラム側で合算するロジックを実装することが一般的である。合算の対象となる主なタグは、jppfs_cor:ShortTermLoansPayable、jppfs_cor:LongTermLoansPayable、jppfs_cor:BondsPayable、jppfs_cor:CurrentPortionOfLongTermLoansPayable などである（信頼度レベル：高）。

Node.js/TypeScript でのXBRL解析

Node.jsおよびTypeScript環境において、EDINETから取得したZIPファイルを解凍し、XBRL/iXBRLを解析するパイプラインを構築するにあたっては、ライブラリの選定がパフォーマンスや保守性を大きく左右する。

ライブラリ分類	候補ライブラリ	評価および推奨される理由
ZIP展開	jszip, adm-zip, unzipper	サーバーレス環境（Vercel等）におけるメモリ効率や非同期処理の扱いやすさを考慮すると、ブラウザ・Node.js 双方で動作しメモリ上で展開が完結しやすい jszip の採用

		が推奨される。
XMLパーサー	fast-xml-parser, xml2js, saxes	fast-xml-parser はその名の通り非常に高速であり、かつ TypeScriptとの相性も良いため、近年のNode.jsにおける大規模XML解析のデファクトスタンダードになりつつある ²¹ 。
iXBRLパーサー	cheerio	iXBRLは本質的にHTML文書であるため、XMLパーサーではなくスクレイピングライブラリである cheerio を用いることで、jQueryライクなセレクトによる要素抽出が可能となる ¹⁷ 。

さらに、抽出された数値データに対する正規化の処理は非常に重要である²²。有価証券報告書内では、財務数値が全角文字で記載されていたり、マイナスを表現するために「△」「▲」「-」「ー」「括弧()」といった多様な記法が混在して用いられることがある²²。また、パーセント(%)が含まれている場合は最後に 10^{-2} を乗じるなどの処理が必要となる²²。これらを一律にJavaScriptの Number 型として扱える半角数字と標準的なマイナス記号に置換する正規化ロジックを自作する必要がある²²。

以下に、ZIP展開からiXBRLファイルの判定、パース、および値の抽出を行う一連のフローを実装したTypeScriptのコード例を示す。

TypeScript

```
import * as jszip from 'jszip';
import * as cheerio from 'cheerio';
```

```
interface FinancialData {
  sales?: number;
  operatingProfit?: number;
}
```

```
function normalizeFinancialValue(rawText: string, scale: string, sign?: string): number {
  if (!rawText |
```

```

| rawText.trim() === "") return 0;

// 全角数字を半角に変換し、カンマを除去
let valueStr = rawText.replace(/[0-9]/g, (s) => String.fromCharCode(s.charCodeAt(0) - 0xfee0))
    .replace(/,/g, "")
    .trim();

// 5種類のマイナス表記や括弧表記の判定
const isNegative = valueStr.includes('△') |

|
    valueStr.includes('▲') |

|
    (valueStr.startsWith('(') && valueStr.endsWith('')) ||
    valueStr.startsWith('-') ||
    valueStr.startsWith('−') ||
    sign === '-';

const isPercent = valueStr.includes('%');

// 記号の除去
valueStr = valueStr.replace(/[\(\)\%\-\+]/g, "");

let value = parseFloat(valueStr);
if (isNaN(value)) return 0;

// scale属性の考慮
const scaleInt = parseInt(scale, 10);
if (!isNaN(scaleInt)) {
    value = value * Math.pow(10, scaleInt);
}

if (isPercent) {
    value = value / 100;
}

return isNegative? -value : value;
}

async function parseXbrlZip(zipBuffer: Buffer): Promise<FinancialData> {
    const zip = await jszip.loadAsync(zipBuffer);

```

```

const data: FinancialData = {};

const fileNames = Object.keys(zip.files);
const targetFile = fileNames.find(name => name.includes('PublicDoc') && name.endsWith('.htm'));

if (!targetFile) {
  throw new Error('No appropriate iXBRL file found in PublicDoc directory');
}

const htmContent = await zip.files[targetFile].async('string');
const $ = cheerio.load(htmContent);

$('ix\\:nonFraction').each( (_, element) => {
  const name = $(element).attr('name');
  const contextRef = $(element).attr('contextRef') |

  | ";
  const scale = $(element).attr('scale') |

  | '0';
  const sign = $(element).attr('sign');
  const rawText = $(element).text();

  // 当期かつ連結データの判定(簡易版)
  const isCurrent = contextRef.includes('CurrentYear') |

  | contextRef.includes('CurrentDuration');
  const isConsolidated = !contextRef.includes('NonConsolidatedMember');

  if (isCurrent && isConsolidated) {
    if (name === 'jppfs_cor:NetSales' |

  | name === 'jpigp_cor:RevenueIFRS') {
      data.sales = normalizeFinancialValue(rawText, scale, sign);
    } else if (name === 'jppfs_cor:OperatingIncome' |

  | name === 'jpigp_cor:OperatingProfitLossIFRS') {
      data.operatingProfit = normalizeFinancialValue(rawText, scale, sign);
    }
  }
});

return data;

```

}

勘定科目マッピング戦略

日本の財務開示において、企業の業態や文化によって同一の財務的意味を持つ項目に異なる勘定科目名が使用される「表記揺れ」は非常に頻繁に発生する¹⁵。これらを適切に正規の項目にマッピングすることは、自動化された財務分析アプリにおいてデータ品質を維持するための生命線である。

財務的意味	実際に使用される勘定科目名の例(表記揺れの具体例10選)
売上高	売上高, 営業収益, 役務収益, 完成工事高, 営業収入, 保険料等収入, 営業収益合計, 収益合計
営業利益	営業利益, 営業損失, 営業利益(△損失), 営業評価損益前利益

これらを適切に処理するため、まずは書類内の `jpdei_cor:AccountingStandardsDEI` タグを参照して、その企業がどの会計基準を採用しているかを判定することが基礎となる¹⁴。その上で、「優先順位付きフォールバック検索」のアルゴリズムを設計することが望ましい。すなわち、まずは標準的な `NetSales` 等のタグを探し、存在しない場合は `Revenues` を探し、それでも存在しない場合は特定の業種固有のタグを探索する、といった順序付きの探索リストを配列として保持する設計である。

こうしたマッピングテーブルを管理する際、TypeScriptにおける型定義は以下のようなデータ構造が推奨される。

TypeScript

```
type AccountingStandard = 'J-GAAP' | 'IFRS' | 'US-GAAP';

interface TagMapping {
  standard: AccountingStandard;
  canonicalName: string;
  primaryTag: string;
  fallbackTags: string;
```

```
}
```

```
const salesMapping: TagMapping = {  
  standard: 'J-GAAP',  
  canonicalName: 'sales',  
  primaryTag: 'jppfs_cor:NetSales',  
  fallbackTags:  
};
```

こうした設計において、PythonのOSSである edinet-mcp が採用している taxonomy.yaml アプローチ(会計基準ごとのマッピングや表記揺れを1つの静的ファイルで一元管理するアプローチ)は非常に洗練されており、TypeScriptの環境下においても同様の静的定義ファイル(JSONやYAML)を用意して読み込ませるアーキテクチャを採用することが極めて合理的である²³。

既存OSSの評価

有価証券報告書やXBRLの解析に利用可能な既存のオープンソースソフトウェア(OSS)を調査し、今回のNext.js環境への適用可能性について比較検討を行った。

プロジェクト名	言語・特徴	StockManagementプロジェクトへの適用可能性評価
edinet-mcp	Python ²³	161個の正規化ラベルと26の財務指標算出機能を持ち非常に強力だが、Node.js環境から直接呼び出すことはできず、サーバーレス環境での連携にはPython APIを別途構築する必要がある ²³ 。
edinet-xbrl	Python ²⁴	EDINETに特化したパーサーだが、やはり実行環境がPythonであるため、本プロジェクトのVercelネイティブな環境には統合しづらい ²⁴ 。
Arelle	Python	XBRLの国際標準パーサーであり、極めて高い準拠性を持つ。しかしながら、非常に重量級のアプリケーションであり、Vercelのサーバーレス関数上で動かすには完全に不

		向きである。
dexter-jp / edinetdb	TypeScript ²⁵	TypeScript製で開発されており、Next.jsとの親和性が極めて高い ²⁵ 。本プロジェクトに最も近い技術スタックを持つ。
edinet-tools	Python ⁶	EDINETデータ用のSDKであり、型定義されたパーサーを提供するが、Node.js環境への移植が必要となる ⁶ 。

Vercel サーバーレス関数の制約

本プロジェクトにおいて、VercelのHobbyプラン(無料枠)を利用してXBRLの取得およびパースを実行する場合、いくつかの極めて厳しい技術的制約に直面することになる²⁶。

まず、Hobbyプランにおけるサーバーレス関数のデフォルトタイムアウトは10秒に設定されている点である(Fluid Compute等を有効化することで最大60秒の実行が可能となる場合もある)²⁷。EDINET APIからのZIPファイルのダウンロード、ZIPのメモリ上での展開、そして数メガバイトに及ぶiXBRL(HTML)テキストをCheerio等でパースして全タグを走査する一連の処理は、データ量の大きい企業の報告書(例えば、金融機関や大規模な多国籍企業など、ZIPサイズが10MBに達することもある)においては、10秒以内に完結しない可能性が非常に高い⁷。

さらに、メモリの制約はHobbyプランで最大2GB(2048MB)と定義されているものの²⁶、ZIPをメモリ上で一気に展開し、DOMツリーを構築しようとする、メモリ不足によるクラッシュや実行性能の低下を引き起こす懸念もある。また、Vercelの関数実行における最大リクエストおよびレスポンスペイロードサイズは4.5MBに制限されているため、直接的に巨大なZIPファイルを関数の返却値として扱うことはできない²⁶。

これらの制約を回避するための現実的な代替案として、Supabase Edge Functionsの活用が強く推奨される³⁰。Supabase Edge FunctionsはDenoランタイムで動作し、無料枠であっても最大150秒程度の長時間の実行が許容される(信頼度レベル:高)。したがって、Next.jsのAPIルートからはトリガーだけを発行し、実際の「EDINETからのZIPダウンロード」「XBRLパース」「Supabaseへの保存」というヘビーなバッチ処理はSupabase Edge Functions側で行うように責任を分離するアーキテクチャが最適である³⁰。また、定期的に新しい報告書を巡回・取得する機能については、Vercel Cronなどを利用してバックグラウンドで発火させることが現実解となる。

CSV(type=5) vs XBRLパースの比較

EDINET API v2から提供が開始された type=5(CSV形式)のデータと、従来のZIP内のXBRLファイルを自前でパースする方法について、それぞれの特性を以下の通り比較する³。

比較項目	CSV形式 (type=5)	自前でのXBRLパース
取得可能な項目	表紙情報や主要な財務指標サマリー等の特定の項目に限定される ³	有価証券報告書内に埋め込まれた全ての数値・テキストブロックが取得可能
解析の難易度	カンマ区切りのテキストであるため、Node.js標準のパースャで容易に処理可能	XML/iXBRLの高度な知識、コンテキストの解釈、scaleの考慮等が必要で難解
データ量	ファイルサイズが非常に小さく、Vercelのタイムアウトやメモリ制約に引っかかりにくい	ZIPファイルが数MB~10MB超になることがあり、サーバーレス環境では負荷が高い
CSVにない項目	取得不可 (詳細な注記、セグメント情報、キャッシュ・フローの詳細など)	取得可能 (完全な開示情報が手に入る)

CSVの列構造は、勘定科目の要素名、項目名(日本語)、当期金額、前期金額などがシンプルに並んだ構造をとるため、csv-parserなどの標準的なNode.jsライブラリで容易にパースが可能である。株式投資分析アプリの第一歩として、まずは実装コストが極めて低く、タイムアウトの心配も少ないtype=5のCSV形式による主要財務項目の自動取得から着手することを強く推奨する。

StockManagementプロジェクトへの推奨実装ロードマップ

これまで述べてきた制約や技術的評価を総合し、今回構築する株式投資財務分析Webアプリ(StockManagement)を、VercelおよびSupabaseの無料枠(月額¥0)の範囲内で堅牢に実現するための具体的な実装ロードマップを提示する。

フェーズ1: 基盤構築とCSVによるMVP開発

プロジェクトの初期段階においては、Vercel Cronを活用して定期的に書類一覧API(/api/v2/documents.json)を呼び出し、当日に提出された有価証券報告書を特定する仕組みをNext.js上に構築することが望ましい¹。ここで取得する書類データは、実装の手間やタイムアウトのリスクを最小化するため、type=5のCSV形式に限定する³。これにより、Vercel Hobbyプランにおける制限時間の壁に衝突することなく、主要財務指標をSupabaseのデータベースに格納する仕組みを迅速に構築することができる²⁷。まずはアプリの最小実行単位を形にすることが優先される。

フェーズ2: Supabase Edge Functions を用いた本格的なXBRL解析への移行

CSV形式から得られる限定的な項目のみでは、アプリとしての差別化やディープな財務スクリーニングを実現することは難しいため、次段階として本質的なXBRL/iXBRL解析エンジンの構築へ移行す

る。ここではVercelのタイムアウトを回避するため、ZIPダウンロードからパース、データ保存までの一連の重たい処理をSupabase Edge Functionsへとオフロードするアーキテクチャを採用する³⁰。Deno環境上で動くこの関数群は、無料枠であっても150秒程度の実行時間が許容されるため、サイズの大きい有報ZIPであっても安全に処理を完遂させることが可能となる。

フェーズ3: 勘定科目マッピングの高度化とAI連携

最終フェーズとして、各企業や会計基準ごとに散在する勘定科目名の揺れを静的定義ファイルによって高度に抽象化し、正規化されたデータモデルへとマッピングするレイヤーを完成させる。これにより、J-GAAP、IFRS、US-GAAPが混在する日本の株式市場においても、統一的な尺度で企業間比較を行える環境が整う。将来的には、抽出した非財務情報(テキストブロック)を大規模言語モデルに流し込み、企業の定性的なリスクや競争優位性を自動で評価するような、AIを活用した金融リサーチエージェントへとアプリを発展させていくことが可能となる²⁵。

引用文献

1. 【Python】EDINETのAPI v2を使って決算書類データをスクレイピングするサンプルコード - note, 4月 4, 2026にアクセス、https://note.com/python_lab/n/n510c3315b3f9
2. 有価証券報告書をpythonで取得してみる2024 金融庁API版 - Qiita, 4月 4, 2026にアクセス、<https://qiita.com/YoshitakaOkada/items/877a331a450a8f27f985>
3. Q1.『書類閲覧利用環境』に記載がないOSやWebブラウザなのですが、閲覧はできますか。 - EDINET, 4月 4, 2026にアクセス、https://disclosure2dl.edinet-fsa.go.jp/guide/static/disclosure/WZEK0090_001.html
4. EDINET APIを利用して、企業情報(XBRLデータ)を自動で集めてみよう(4/10) - Qiita, 4月 4, 2026にアクセス、<https://qiita.com/XBRLJapan/items/27e623b8ca871740f352>
5. edinet-api-tools/edinet_api_parse_sec_report.py at master - GitHub, 4月 4, 2026にアクセス、https://github.com/sarubee/edinet-api-tools/blob/master/edinet_api_parse_sec_report.py
6. matthelmer/edinet-tools: Python SDK for Japanese corporate disclosure data - GitHub, 4月 4, 2026にアクセス、<https://github.com/matthelmer/edinet-api-tools>
7. 決算短信 提出書類ファイル仕様書(2024 年 7 月対応) - JPX, 4月 4, 2026にアクセス、https://www.jpx.co.jp/equities/listing/disclosure/xbml/nlsgeu000005vk0b-att/File_Specification_for_TDnet_Filing.pdf
8. 報告書(XBRL)作成 ツール 操作ガイド - 金融庁, 4月 4, 2026にアクセス、<https://www.fsa.go.jp/search/20130917/04.pdf>
9. 報告書(XBRL)作成 ツール 操作ガイド - EDINET, 4月 4, 2026にアクセス、<https://disclosure2dl.edinet-fsa.go.jp/guide/static/submit/download/ESE140107.pdf>
10. Application of iXBRL in South Africa Introductory training Maphuthumane Masilela (CIPC) Michal Zubrycki (BR-AG), 4月 4, 2026にアクセス、https://www.cipc.co.za/wp-content/uploads/2021/04/XBRL_SA_Introductory_Training.pdf
11. 報告書インスタンス 作成ガイドライン - 金融庁, 4月 4, 2026にアクセス、https://www.fsa.go.jp/search/20241112/2b-1_InstanceGuide.pdf

12. 添付 1 物理ファイル一覧, 4月 4, 2026にアクセス、
https://www.fsa.go.jp/search/20180316/1h_Sample_Setsumeitenpu.xlsx
13. ATCRP - JPX, 4月 4, 2026にアクセス、
https://www.jpx.co.jp/equities/listing/disclosure/xbrl/nlsgeu000005vk0b-att/koumoku_list_Quarterly_Financial_Statements_20250131.xlsx
14. EDINETタクソノミ更新概要添付資料 (EXCEL:163KB), 4月 4, 2026にアクセス、
https://www.fsa.go.jp/search/20241112/1a-2_KoshinGaiyo_Tenpu.xlsx
15. xbrl_parser/xbrl_parser_for_segment.py at master - GitHub, 4月 4, 2026にアクセス、
https://github.com/miya8/xbrl_parser/blob/master/xbrl_parser_for_segment.py
16. StreamlitでEDINETから有価証券報告書をダウンロードして分析するWEBアプリをサクッとつくろう, 4月 4, 2026にアクセス、
<https://zenn.dev/akb428/articles/7c1c05d3cc1e21>
17. How to get contextRef text of the fact? · manusimdt py-xbrl · Discussion #129 - GitHub, 4月 4, 2026にアクセス、
<https://github.com/manusimdt/py-xbrl/discussions/129>
18. Beautiful SoupでEDINETのXBRLから財務諸表のデータベースを構築する #Python - Qiita, 4月 4, 2026にアクセス、
<https://qiita.com/XBRLJapan/items/d23bc251c53d81d49852>
19. Duplicate Facts - Certent - dm, 4月 4, 2026にアクセス、
https://dm.certent.com/help/Content/I_XBRL/ixbrl/duplicate_facts.htm
20. How to Design Taxonomies for Accurate Unit Declarations - XBRL International, 4月 4, 2026にアクセス、
<https://www.xbrl.org/guidance/unit-declarations/>
21. edinet-ts/docs/REQUIREMENTS.md at master - GitHub, 4月 4, 2026にアクセス、
<https://github.com/oharato/edinet-ts/blob/master/docs/REQUIREMENTS.md>
22. 決算短信のXBRLを徹底解剖！Google Colabでの財務データ抽出ステップ - Qiita, 4月 4, 2026にアクセス、
<https://qiita.com/maskot1977/items/ebb1c804d69f2dbe58b0>
23. ajtgjmdjp/edinet-mcp: EDINET XBRL parsing library and ... - GitHub, 4月 4, 2026にアクセス、
<https://github.com/ajtgjmdjp/edinet-mcp>
24. ajtgjmdjp/awesome-japan-finance-data: A curated list of open data sources, APIs, Python libraries, and MCP tools for Japanese financial and economic data - GitHub, 4月 4, 2026にアクセス、
<https://github.com/ajtgjmdjp/awesome-japan-finance-data>
25. EDINET DB edinetdb - GitHub, 4月 4, 2026にアクセス、
<https://github.com/edinetdb>
26. Vercel Functions Limits, 4月 4, 2026にアクセス、
<https://vercel.com/docs/functions/limitations>
27. Limits - Vercel, 4月 4, 2026にアクセス、
<https://vercel.com/docs/limits>
28. Vercel Hobby Plan, 4月 4, 2026にアクセス、
<https://vercel.com/docs/plans/hobby>
29. What can I do about Vercel Functions timing out?, 4月 4, 2026にアクセス、
<https://vercel.com/kb/guide/what-can-i-do-about-vercel-serverless-functions-timing-out>
30. Vercelのタイムアウト問題を解決できますか？ : r/nextjs - Reddit, 4月 4, 2026にアクセス、
https://www.reddit.com/r/nextjs/comments/1aim533/are_you_able_to_solve_the_ti

[meout_issue_in_vercel/?tl=ja](#)